LLM Agents

Autonomous Software Agents

A.A. 2025-2026

Prof. Paolo Giorgini



UNIVERSITY OF TRENTO - Italy

Department of Information
and Communication Technology

LLM vs LLM agents

LLM

- *Responds to a prompt by generating text*
- Produces explanations, suggestions, or information

LLM Agent

- *Acts in the external world to achieve a goal specified in the prompt*
- It can plan, use tools (APIs, databases, software), and execute actions

---

Prompt: *"Organize a trip to Rome for tomorrow with a budget of €150"*

- Suggests possible trains
- Recommends budget hotels
- Proposes a travel itinerary

→ **Provides information**

- Searches train availability via APIs
- Compares hotel prices
- Books the train and hotel
- Sends a confirmation email

→ **Performs actions to achieve the goal**

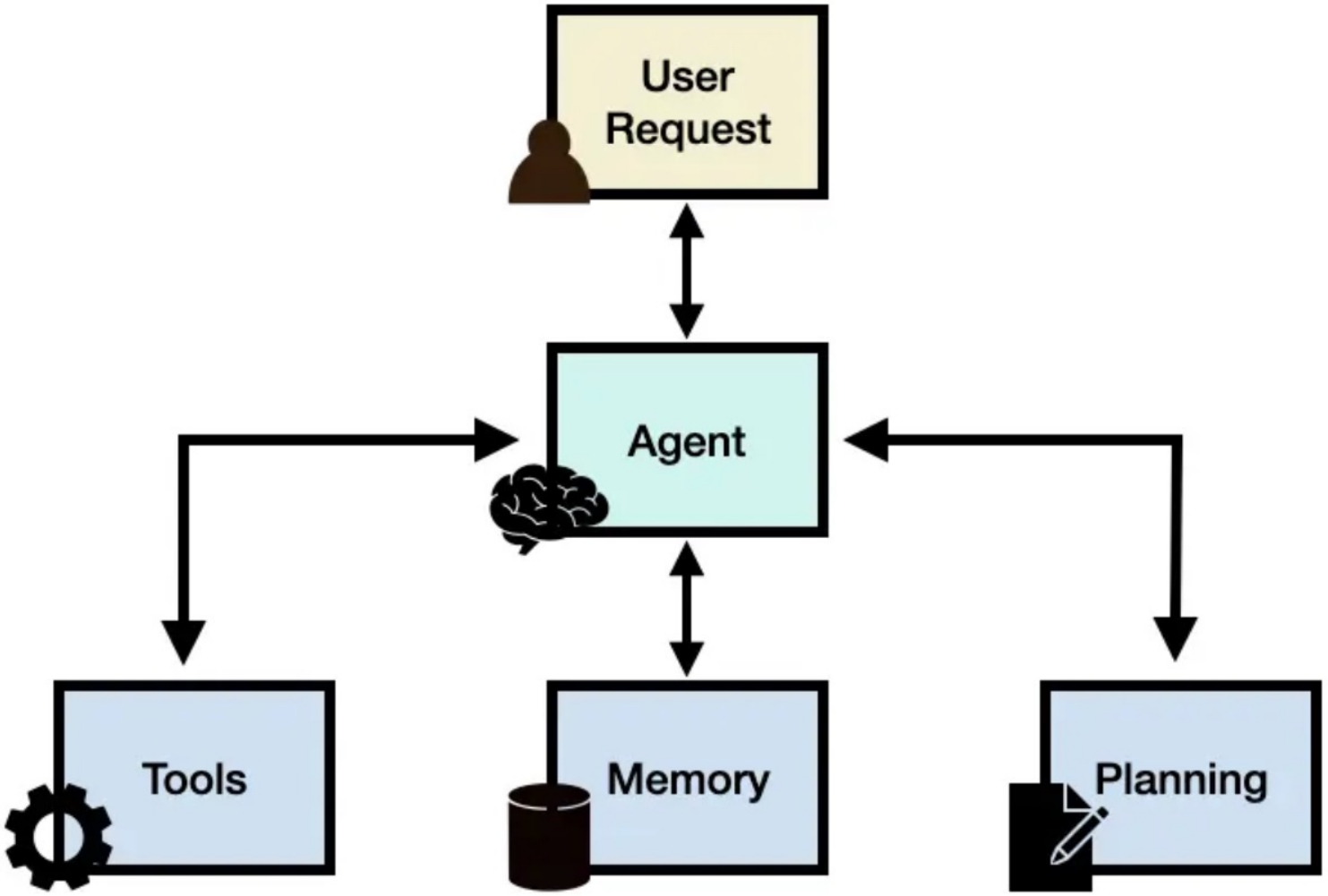
“LLM responds.
LLM Agent acts”

What are LLM agents?

- LLM-based agents (LLM agents) are **AI applications** that execute complex tasks using LLMs combined with **modules** like **planning**, **memory**, and **tool usage**
- **Core components**
  - **LLM as the Controller** – Acts as the 'brain' to guide operations
  - **Planning Module** – Breaks down tasks into structured steps
  - **Memory Module** – Stores and retrieves past interactions
  - **Tool Usage Module** – Interacts with external tools and APIs

How LLM Agents Work

- The LLM processes user input and determines the required steps
- The agent uses memory to maintain context
- Planning ensures systematic execution of tasks
- The agent leverages external tools when necessary



Agents vs Automation

LLM Agent

- Automates complex workflows efficiently
- Adapts to various tasks with contextual understanding
- Enhances user interactions with advanced reasoning
- Handles tasks that can be done via a screen, significantly expanding automation potential

Agents vs Atomation

- **Agents** make decisions (without manual intervention), adapt, and use memory to handle complex tasks
- **Automations are** predefined workflows that follow fixed rules without adaptability or learning
- Agents can handle dynamic and unpredictable scenarios, while automation is limited to repetitive tasks

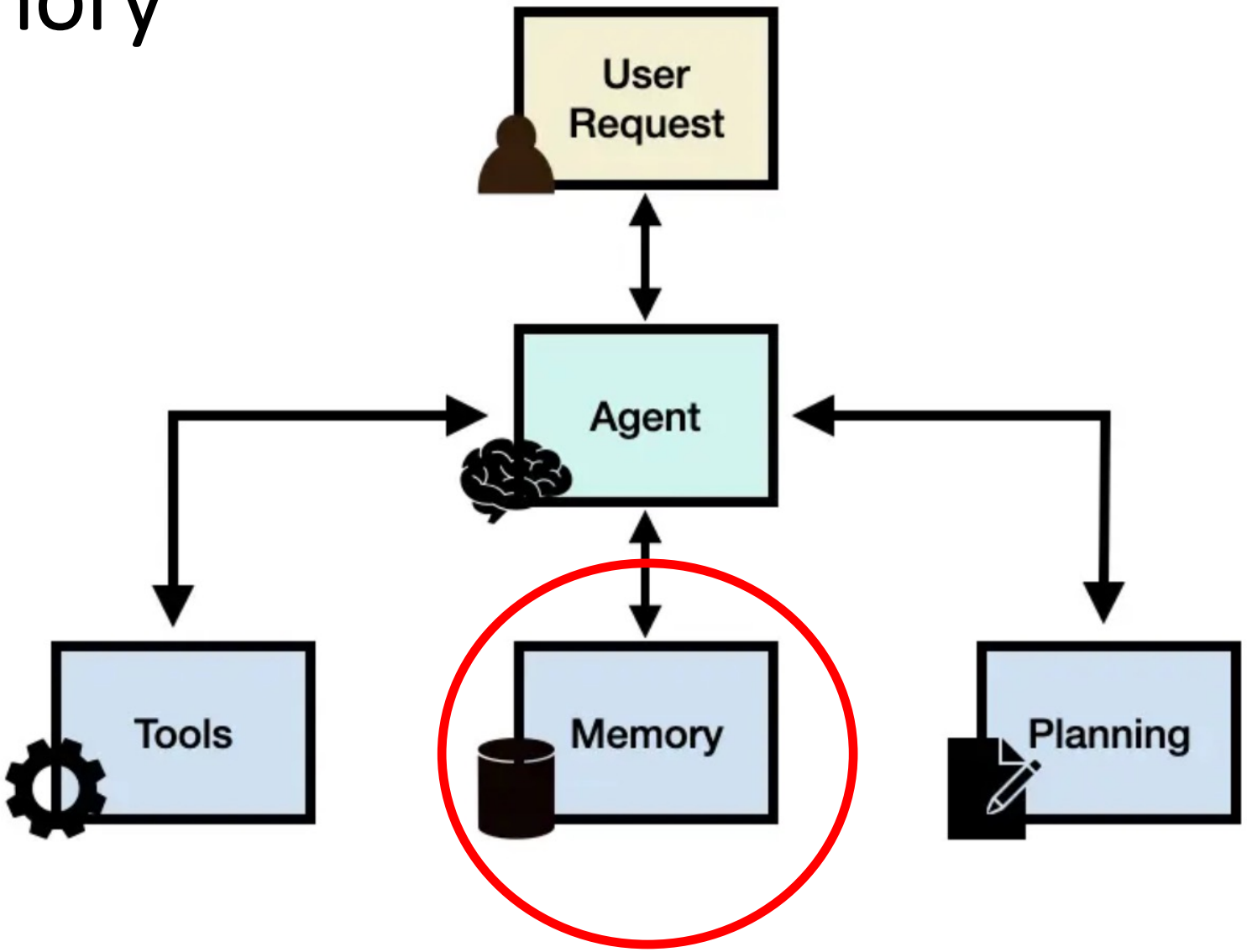
An example of agent vs automation

- Automation tools (e.g., Zappi, n8n) execute predefined workflows
  - If an email asks about ASA class times, automation replies and updates a FAQ
  - If something unexpected happens (e.g., wrong course name), automation fails
- An LLM agent
  - Analyses all incoming emails, extract intent, and check personal knowledge
  - Understands company guidelines, past emails, and writes responses in your tone
  - Can draft a reply for review before sending

Part 1

Memory

Memory



What Is Memory in AI Agents

- Core component of agentic systems, enabling scalability, consistency, personalization, and efficient reasoning beyond the context window
  - mechanisms that allow a model to **retain and reuse information**
- Memory is especially important for:
  - **Knowledge-rich applications**
    - Large amounts of information (documents, databases, external content)
    - Memory is used to **retrieve and maintain relevant context** across a wide information space
  - **Multi-step systems** such as RAG and agents
    - Memory is used to **track previous actions** (e.g., queries, intermediate results, decisions) and ensure coherent progression of the process.
- In agents, memory is used to store:
  - Instructions, examples, context, tool inventories, plans, tool outputs, reflections
- Any AI application that must retain information benefits from memory

The Three Memory Mechanisms

- An AI model typically relies on **three memory mechanisms**:
  - **Internal knowledge**
  - **Short-term memory**
  - **Long-term memory**
- These mechanisms differ in:
  - persistence
  - capacity
  - cost
  - update strategy

Internal Knowledge

- Internal knowledge is stored **inside the model parameters**
  - It comes from **training** and **fine-tuning**
- Properties:
  - Always available
  - Fast access
- Does **not change** unless the model is updated
- Used for information essential to **all tasks**

Short-Term Memory (Context)

- The model's **context window** acts as short-term memory
- Includes:
  - previous conversation turns
  - intermediate reasoning
  - Task-specific information
- Properties:
  - Fast to access
  - Limited capacity
  - Does not persist across tasks
- Best used for **information immediately relevant** to the current task

Long-Term Memory

- Long-term memory is implemented via **external data sources**
  - e.g. vector databases, documents, logs
- Accessed through **retrieval**, as in RAG
- Properties:
  - Persistent across tasks
  - Cheaper and more scalable than context
  - Can be updated or deleted without retraining the model
- Used for information that is **rarely needed**, but important

Memory Hierarchy and Human Analogy

- Memory usage depends on **frequency of use**
  - Essential for all tasks → internal knowledge
  - Task-specific → short-term memory
  - Rarely needed → long-term memory
- Humans use similar mechanisms:
  - Breathing → internal knowledge
  - Name of someone you just met → short-term memory
  - Books, notes, computers → long-term memory
- Agents follow the same hierarchy

Why Memory Systems Are Useful

Augmenting an AI model with memory enables

- **Managing information overflow**
  - Excess information can be moved out of the context window
- **Persistence across sessions**
  - Personalization and continuity (e.g., remembering user preferences)
- **Improved consistency**
  - Referencing past answers helps produce stable outputs

Memory Management and Retrieval

A memory system typically supports two functions

1. **Memory management**
   - Decide what to store
   - Decide what to delete
2. **Memory retrieval**
   - Fetch relevant information for the current task

- Key points:
  - Short-term memory is constrained by context length
  - Long-term memory can absorb overflow
  - A portion of the context is usually reserved for retrieved memory
  - e.g. 30% for long-term retrieval, 70% for short-term context

Memory Management Strategies

- The simplest strategy: **FIFO (First-In, First-Out)**
  - Oldest information is removed first
- Limitations:
  - Early information may still be important
- More advanced strategies:
  - Remove redundancy
  - Summarize conversations
  - Track named entities
- Summaries can be generated by the same or another model

Reflection, Contradictions, and Mem Updates

- Reflection-based memory management

  After each action, the agent:

  1. Reflects on new information
  2. Decides whether to add, merge, or replace memory

- Handling contradictions:
  - Keep the newest information
  - Ask a model to judge which one to keep
- Contradictions can:
  - confuse an agent
  - but also enable multiple perspectives

Retrieval-Augmented Generation (RAG)

RAG enhances generation by retrieving relevant information from external sources

External memory examples

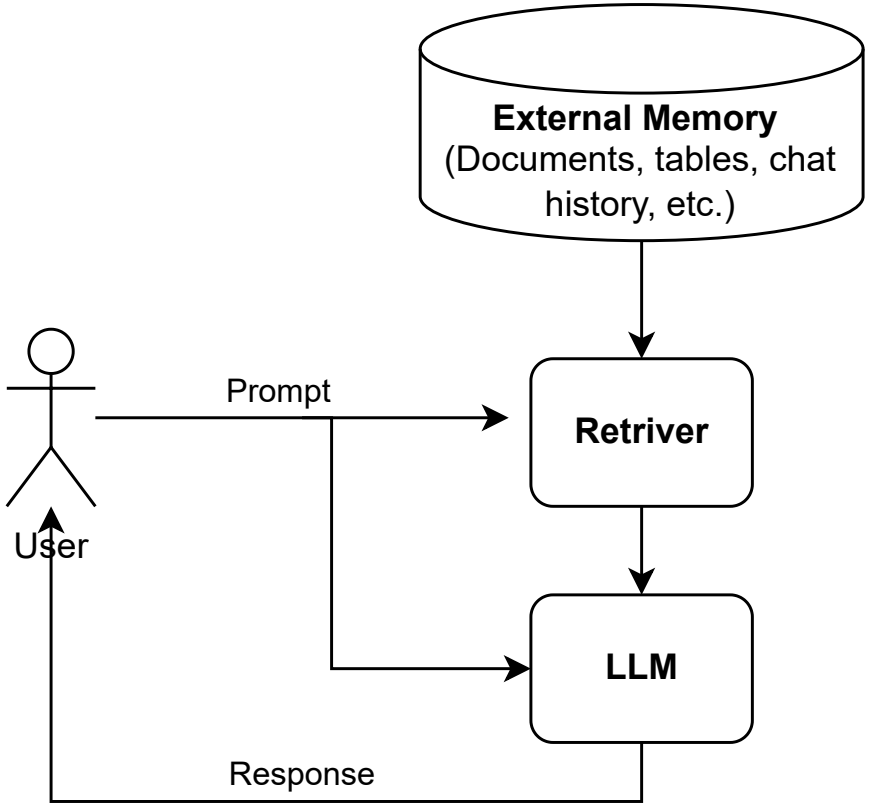
- Document databases
- Knowledge bases
- Chat history
- Internet sources

"What is the company policy for remote work?"

Without RAG:

- The model may hallucinate
- Or give a generic, incorrect policy

RAG Architecture



- **Information Retrieval** is not specific to RAG (**century-old research area)**
  - Backbone of: search engines, recommender systems, log analytics
  - Many retrieval algorithms used in traditional IR systems can be directly reused in RAG architectures
  - **Ranking documents** according to their relevance to a query
  - **Term-based** retrieval
  - **Embedding-based** retrieval
- RAG does not invent retrieval: it reuses decades of IR research. What changes is how retrieval is integrated with generation

Term-Based (Lexical) Retrieval

- Documents are retrieved based on **keyword overlap** with the query
- The more a term appears in a document, the more relevant the document is for that term → **Term Frequency (TF)**

Problem 1: Too many matching documents

- Many documents may contain the same term
- Context window is limited → cannot include all of them
- Heuristic: rank documents by **term frequency**

Problem 2: Not all terms are equally important

- Queries often contain many terms
- Some terms are informative (*"vietnamese", "recipes"*)
- Others are common and uninformative (*"for", "at"*)

IDF, TF-IDF and BM25

- **Inverse Document Frequency (IDF)**: terms that appear in many documents are less informative, Importance of a term is inversely proportional to the number of documents it appears in
  - *Eg.* ***Query****: "Easy-to-follow Italian recipes"*
    - Terms like **"for"** or **"easy"** appear in many documents → they do **not** help discriminate relevant documents
    - Rare terms like **"Italian"** appear in fewer documents → they are **much more informative**
- **TF-IDF**: documents are ranked using a weighted sum of TF × IDF -> *highlights terms that are locally frequent but globally rare, making them effective signals of document relevance*
- **BM25**: Improved version of TF-IDF, normalizes for document length (strong baseline for modern retrieval systems)

Term-Based (Lexical) Retrieval

- **Key limitation:** Lexical overlap ≠ semantic relevance
  - Performs poorly with Paraphrases, Synonyms, and Long documents

  Eg. **Query:** *"How to cook pasta?"*

  Doc A: "This article explains how to cook pasta step by step", shares many words with the query (how, cook, pasta) -> **High lexical overlap**

  Doc B: "To prepare spaghetti, boil water, add salt, and cook until al dente", uses different words (prepare, spaghetti) -> **Low lexical overlap**
  - *Documents can share words without sharing meaning, and share meaning without sharing words*
- Uses an **inverted index** (term → documents): *enabling fast keyword-based retrieval*

What is an Embedding?

- An **embedding** is a vector representation of text, it encodes the **meaning** of the text in a high-dimensional space
  - A NN that maps text into a **fixed-size vector** capturing the semantic meaning of the text

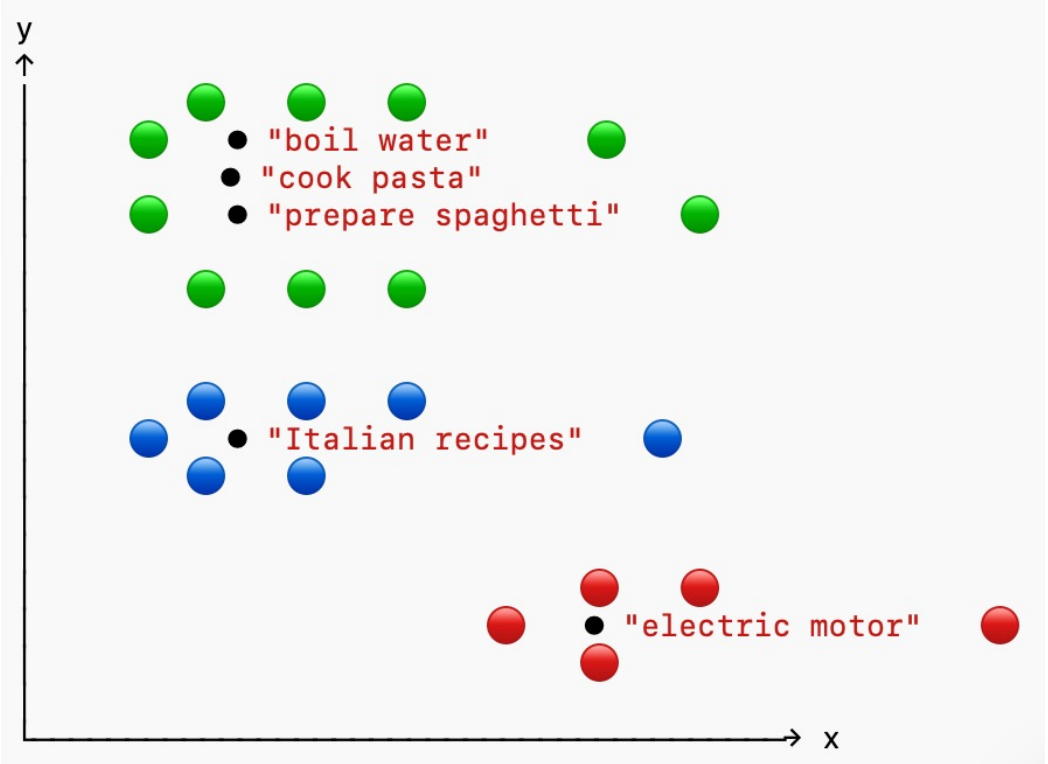
    "cook pasta" → [0.12, -0.34, 0.88, ...]
    "prepare spaghetti" → [0.11, -0.31, 0.86, ...]
    "electric motor" → [-0.72, 0.44, -0.10, ...]

  - Similar meanings → similar vectors ; Dissimilar meanings → distant vectors
  - The model learns to place texts with similar meaning close together
- Examples of embedding models
  - Word-level: Word2Vec, GloVe
  - Sentence / document-level: Sentence-BERT, OpenAI embeddings, Instructor, E5, MPNet

Each dimension in an embedding is a **latent feature learned by the model**; individual dimensions are not directly interpretable, but together they encode semantic meaning through the vector’s position in space

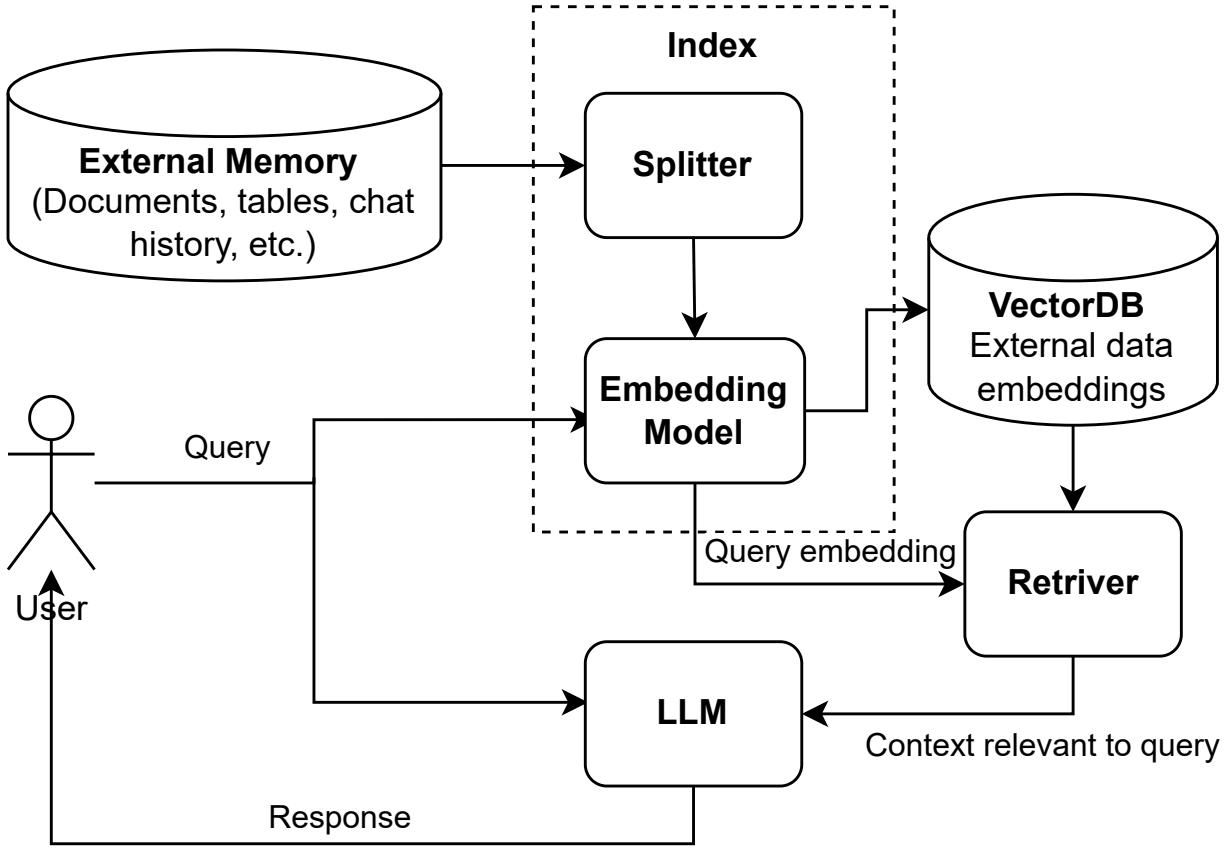
Example: Semantic Similarity with Embeddings

- Query “How to cook pasta?”
  - Doc A “How to cook pasta step by step” → embedding close to query
  - Doc B “How to prepare spaghetti al dente” → embedding also close to query
  - Doc C “History of Italian cuisine” → embedding farther away
- Result
  - Embedding-based retrieval ranks A and B high
  - Term-based retrieval may miss B



We visualize embeddings in 2D, but in practice they live in hundreds of dimensions

How Embedding-Based Retrieval Works



Indexing phase

- Documents are:
  - Split into chunks
  - Converted into embeddings using an embedding model
- Embeddings are stored in a vector DB

Query phase

- The query is converted into an embedding (same embedding model as indexing)
- The retriever finds the k nearest embeddings
- Retrieved chunks are passed to the generative model as context

Vector search

- Typically framed as a nearest-neighbor search (other metrics: Cosine similarity, Euclidean distance, …)

Term-Based vs Embedding-Based Retrieval

Aspect	Term-Based Retrieval	Embedding-Based Retrieval
Matching	Lexical (keyword overlap)	Semantic (meaning similarity)
Query speed	Very fast	Slower (embedding + vector search)
Indexing	Inverted index	Embedding generation + vector index
Performance	Strong out of the box	Improves with tuning
Query expressiveness	Limited	Supports natural language queries
Sensitivity	Term ambiguity, wording	Depends on embedding quality
Interpretability	High	Lower
Cost	Low	Higher (embeddings + storage)
Typical tools	BM25, Elasticsearch	FAISS, ChromaDB, Pinecone
Best use case	Exact matches, IDs, keywords	Semantic search, RAG

Evaluating Retrieval in RAG Systems

- Core Retrieval Metrics
  - **Context precision**: Fraction of retrieved documents that are relevant; measures how clean the retrieved context is
  - **Context recall**: Fraction of relevant documents that are retrieved; measures how complete the retrieved context is
  - High precision avoids noise, high recall avoids missing key information
- What to Evaluate (Three Levels)
  1. **Retriever quality**: Are the right documents retrieved?
  2. **Embedding quality (semantic retrieval)**: Do similar documents have close embeddings? Benchmarks: MTEB (Massive Text Embedding Benchmark)
  3. **End-to-end RAG quality**: Does better retrieval improve final answers?
- Practical Trade-offs
  - **Latency**: Embedding generation + vector search add overhead; Often small compared to generation time
  - **Cost**: Embedding computation; Frequent re-embedding can be expensiveVector storage and ANN indexes (Approximate Nearest Neighbor)

Chunking Strategy

Split documents into chunks to enable effective retrieval

- **Fixed-size chunks**: number of characters, words, sentences, paragraphs
- **Recursive chunking**: Section → paragraph → sentence; reduces loss of related context
- **Token-based chunking**: uses the tokenizer of the generative model; easier downstream integration

Key trade-offs

- **Smaller chunks**: more fine-grained retrieval; higher computational cost
- **Larger chunks**: preserve context; risk of adding noise

Use overlap to avoid cutting important information

There is no universal best chunk size: experimentation is required

Reranking

Improve the quality of retrieved documents after initial retrieval

- **How it works**
  - Step 1: A fast, cheap retriever fetches many candidates
  - Step 2: A slower but more accurate model reranks them (a more accurate model)
- Why reranking matters
  - Helps fit documents into: Limited context window; Token budget; Improves relevance of top documents
- Common reranking criteria: Semantic relevance; Recency (time-based reranking); Task-specific signals

Order matters less than in traditional search, but still affects how well the model uses the context

Query Rewriting

Rewrite user queries to better reflect the actual intent

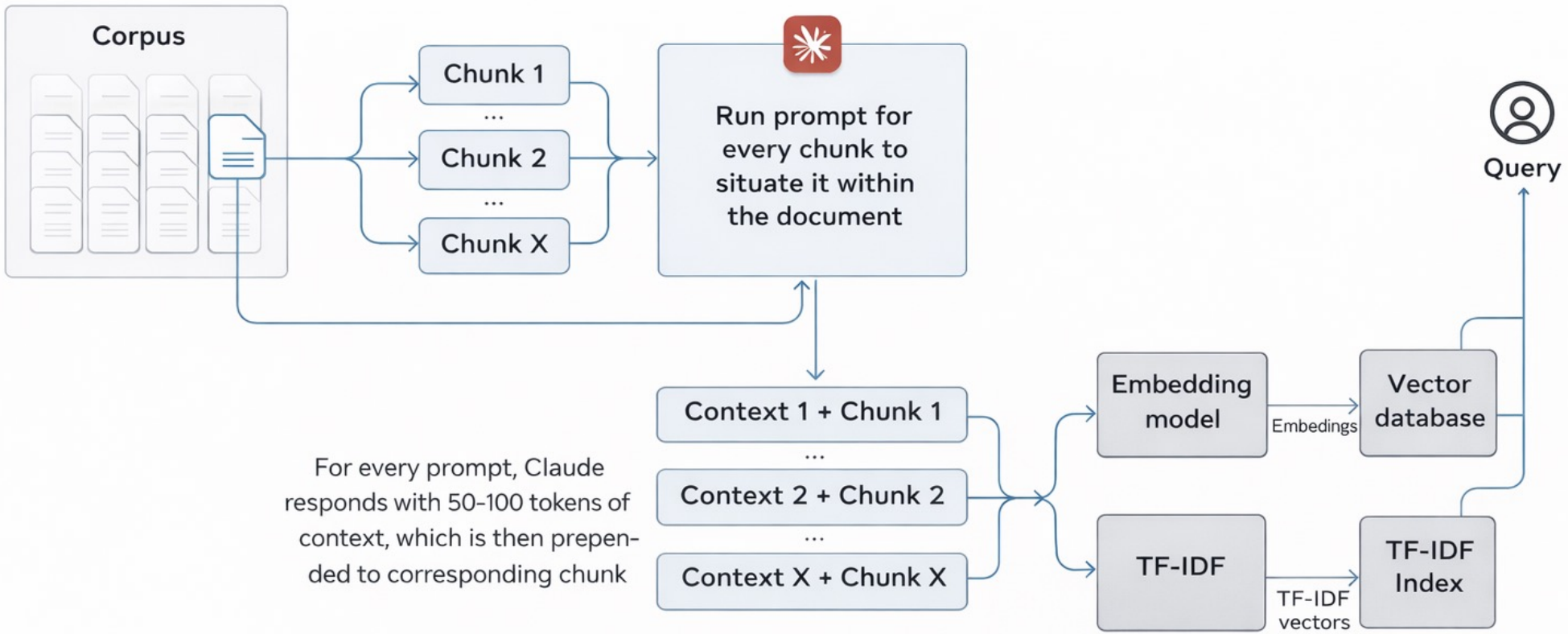
- **Why it's needed:** user queries can be shorts, ambiguous, context-dependent
- E.g., User: "How about Emily Doe?" Rewritten query: "When was the last time Emily Doe bought something from us?"
- **How it's done:** heuristics (traditional search), LLM-based rewriting using conversation context
- **Caution.** Query rewriting may require entity resolution and external knowledge -> If not solvable, the system should avoid hallucinating

Contextual Retrieval

Enrich chunks with additional context to improve retrieval accuracy

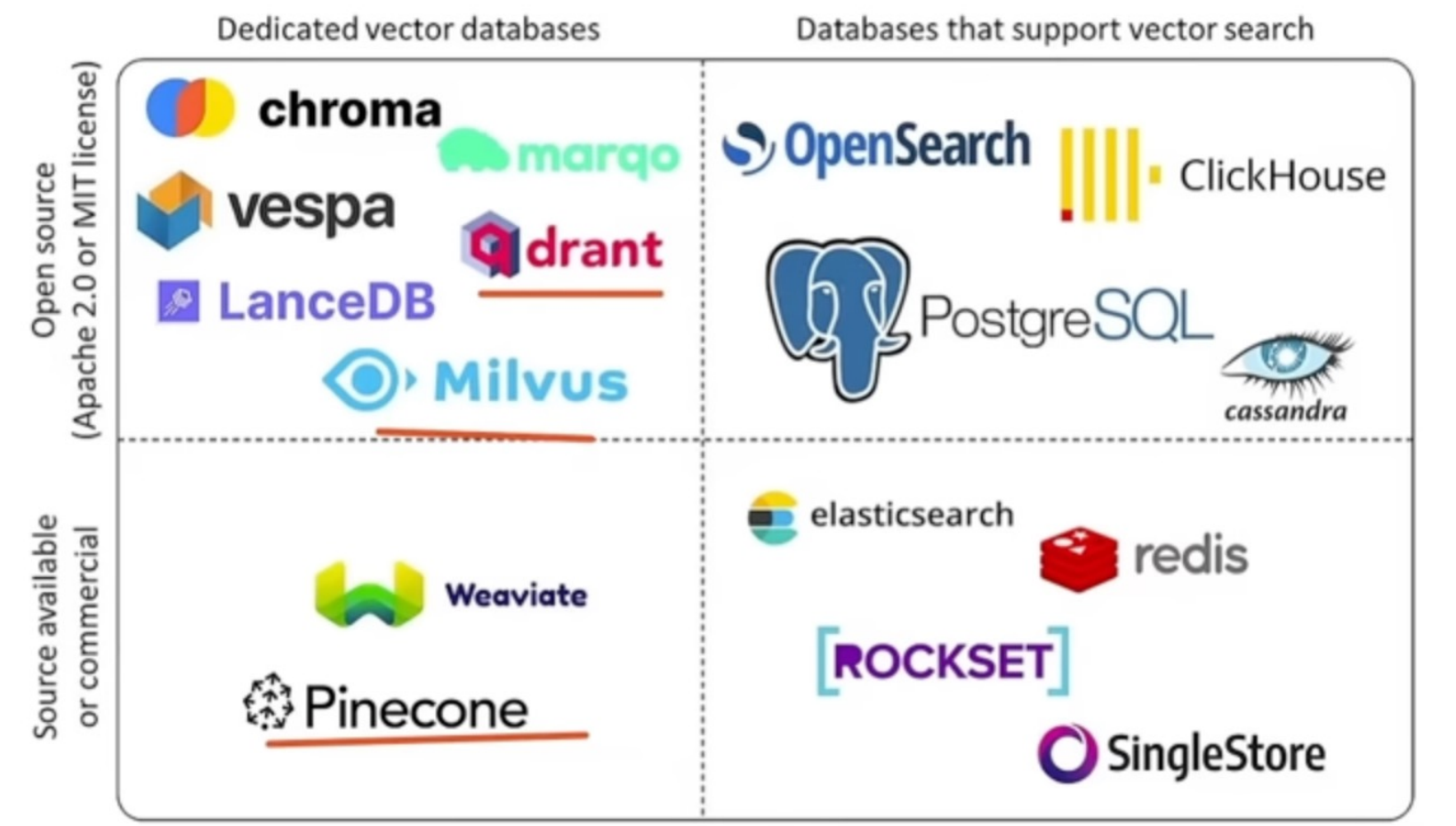
- **How**
  - Add metadata: Tags, keywords, entities
  - Augment chunks with: Document title, Section summary, Short generated context (50–100 tokens)
  - Augment chunks with questions they can answer; particularly effective for: Customer support and FAQ-style data
- **Benefits**
  - Helps retrieve relevant chunks even when: Keywords are missing and chunks are small or ambiguous

PREPROCESSING (new)



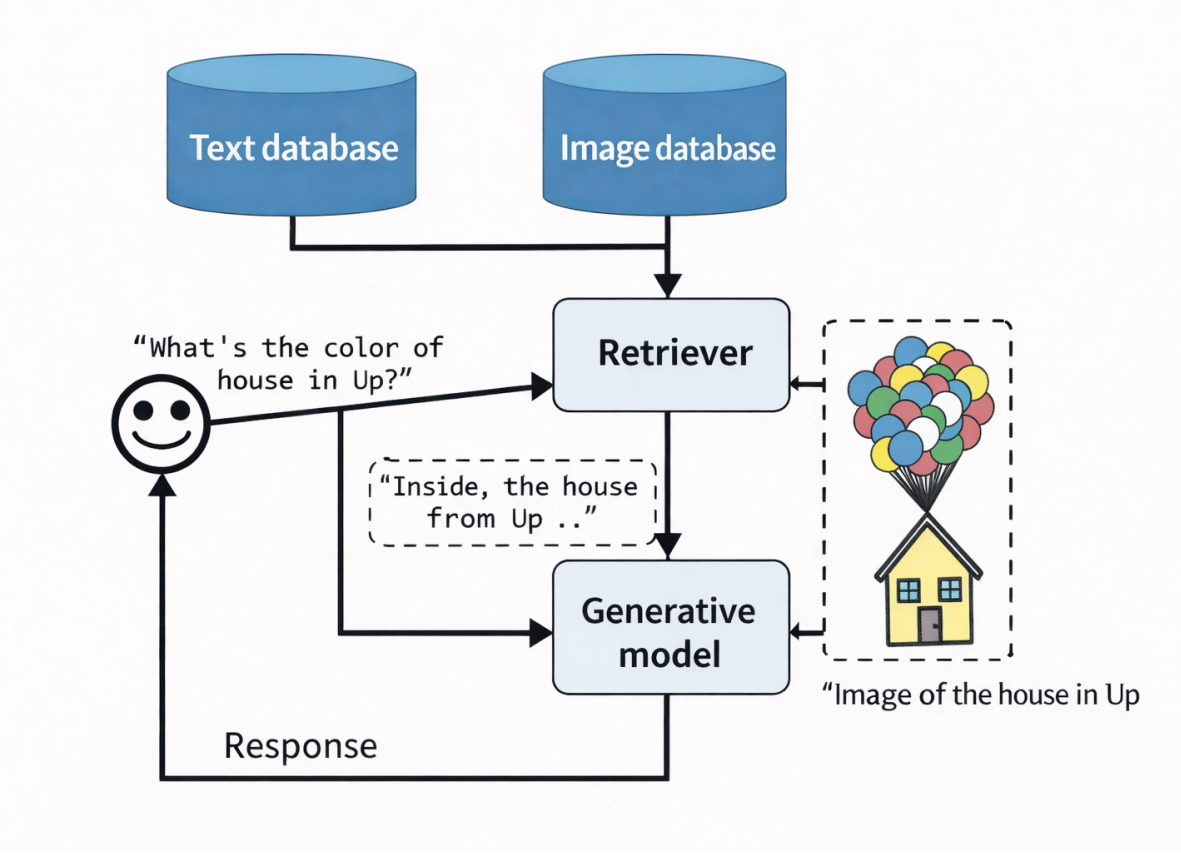
For every prompt, Claude responds with 50–100 tokens of context, which is then prepended to the corresponding chunk

- Small chunks may lack sufficient context
- Augmentation improves:
  - Semantic retrieval
  - Keyword-based retrieval


Dedicated vector databases
Databases that support vector search
Open source (Apache 2.0 or MIT license)
Source available or commercial
chroma
marqo
vespa
qdrant
LanceDB
Milvus
OpenSearch
ClickHouse
PostgreSQL
cassandra
elasticsearch
redis
ROCKSET
SingleStore
Weaviate
Pinecone

RAG Beyond Text: Multimodal RAG

- Classical RAG systems use text documents only, but external data sources can also be images, videos, audio, other multimodal data
- If the generative model is multimodal, **retrieval can also be multimodal**
  - Eg. Query: “What’s the colour of the house in the Pixar movie Up?”
  - The retriever can fetch relevant text descriptions and relevant images to help the model answer correctly



RAG with Tabular Data (Text-to-SQL)

- Many questions cannot be answered by retrieving documents, but require querying structured tables

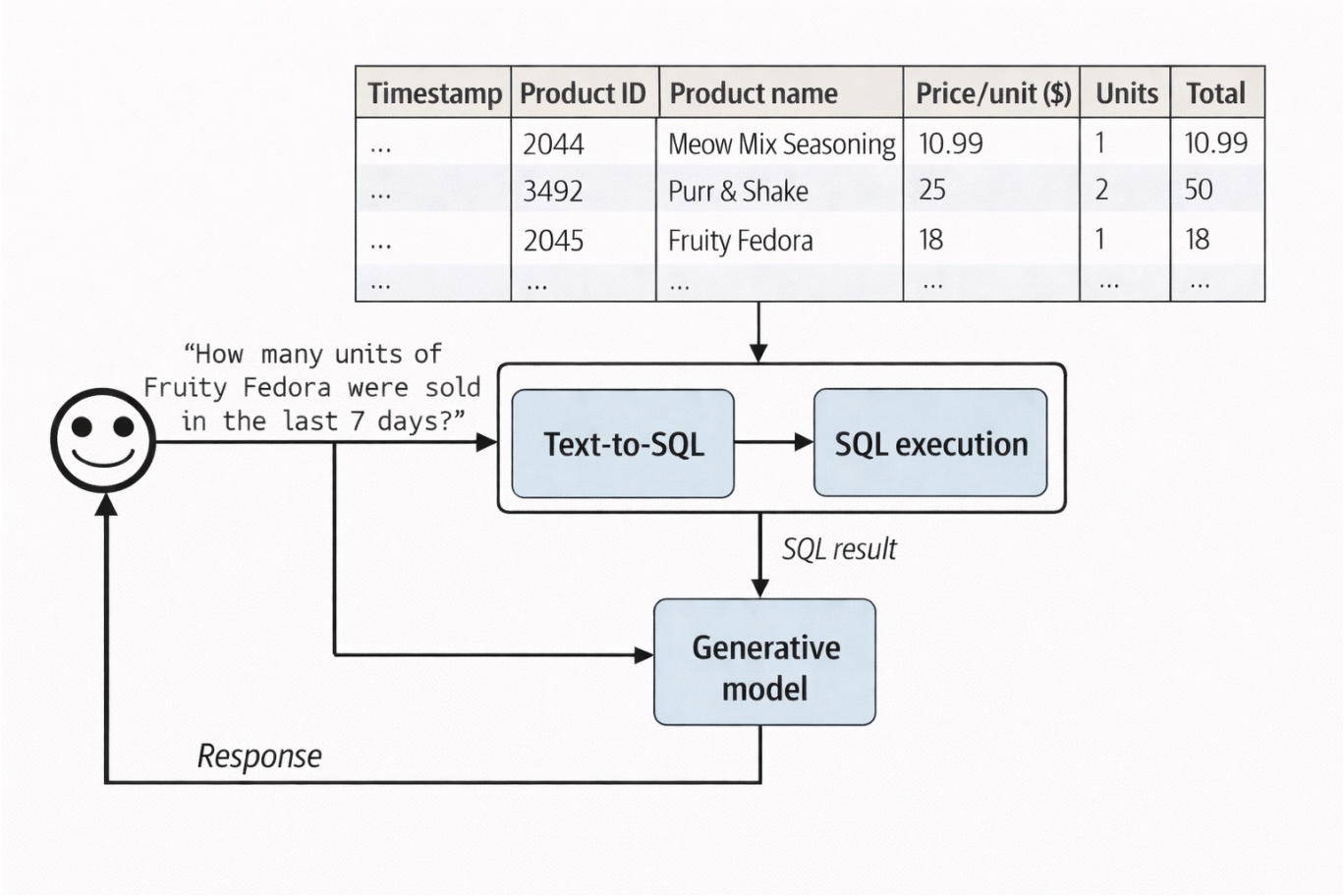
Query: *"How many units of Fruity Fedora were sold in the last 7 days?"*

This requires:

- filtering rows
- aggregating values
- → **SQL**, not text retrieval

Order ID	Timestamp	Product ID	Unit price ($)	Units	Total
1	…	Meow Mix Seasoning	10.99	1	10.99
2	…	Purr & Shake	25	2	50
3	…	Fruity Fedora	18	1	18
…	…	…	…	…	…

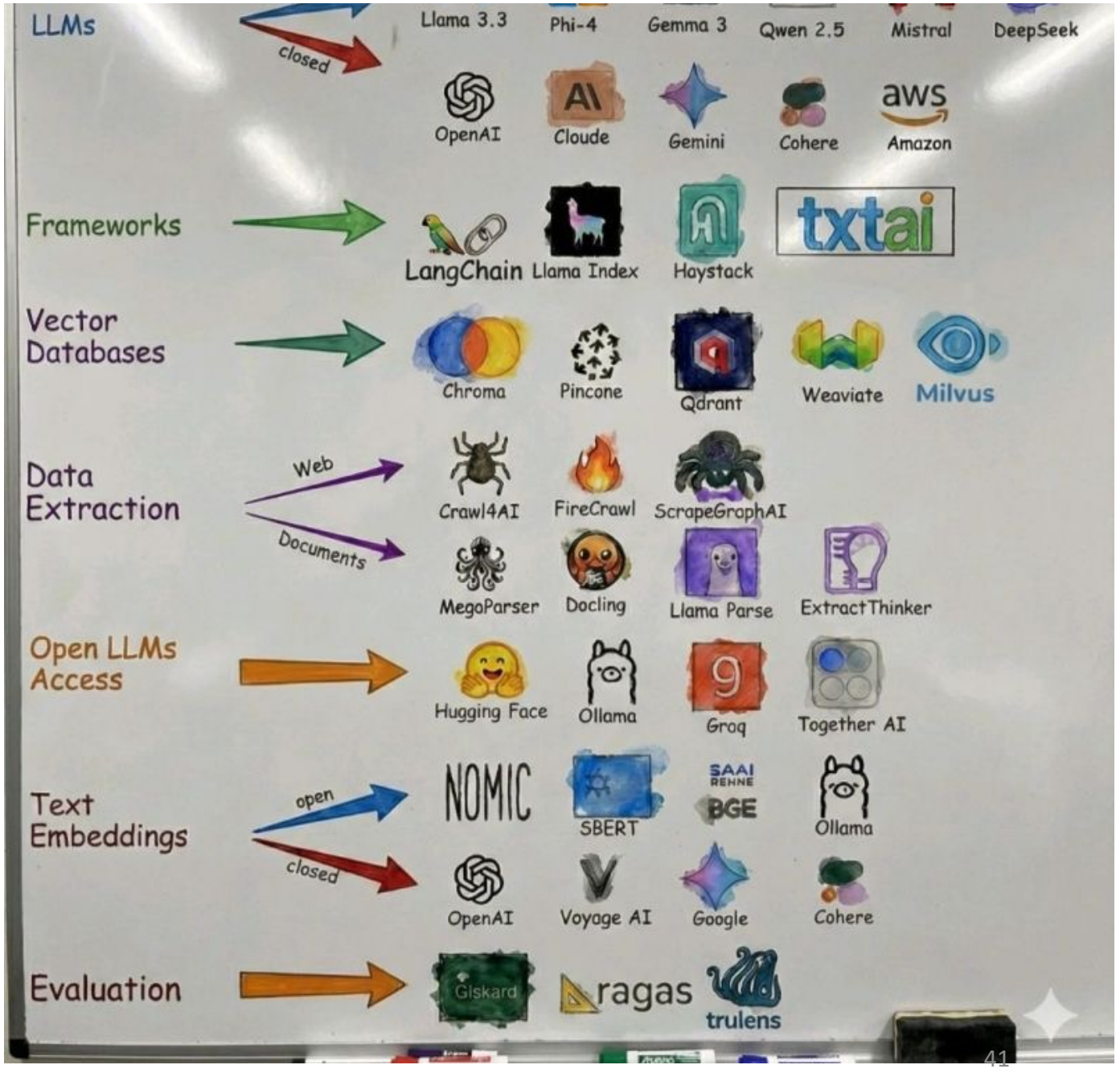
1. **Text-to-SQL**: generate the SQL query
2. **SQL execution**: run the query
3. **Generation**: explain the result in natural language



"How many units of Fruity Fedora were sold in the last 7 days?"

```
SELECT SUM(units) AS total_units_sold
FROM Sales
WHERE product_name = 'Fruity Fedora' AND timestamp >=
DATE_SUB(CURDATE(), INTERVAL 7 DAY);
```

RAG systems are built as layered stacks, combining models, retrieval, data, and evaluation components. Each layer plays a specific role in grounding LLM reasoning with external, up-to-date knowledge.



Summing-up: The typical RAG process

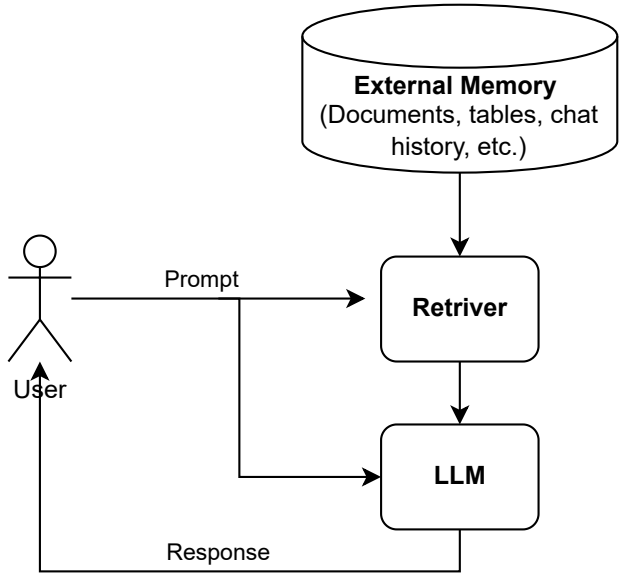
(1) **Preprocessing:** documents are split into chunks (e.g. 1,000 tokens) with metadata (Document title, Section, Date,...)

(2) **Indexing Phase (Offline):** Each chunk is converted into a vector, using an embedding model, which is stored in a vector database (e.g. Chroma)

(3) **User Query Processing (Online).** Query embedding: the query is converted into an embedding (the same model used for documents)

(4) **Retrieval (Vector search):** The system finds the top-k chunks whose embeddings are closest to the query

(5) **Prompt Construction**: chunk 1, 2, 3 (context) … + query



Comparison

Dimension	Prompt Engineering	Context Engineering	RAG (Retrieval-Augmented Generation)
Primary goal	Control model behavior	Control information provided	Ground responses in external knowledge
What you design	Instructions & formatting	Context assembly pipeline	Retrieval + generation pipeline
Input source	Static prompt text	Dynamic runtime context	External documentis via retrieval
When applied	Before inference	At runtime before inference	During inference
Handles large knowledge	❌ No	⚠️ Limited	✅ Yes
Hallucination control	⚠️ Weak-medium	⚠️ Medium	✅ Strong
Best suited for	Task guidance	Agent consistency	Factual accuracy at scale
Best suited for	Task guidance	Agent consistency	Factual accuracy at scale

Prompt engineering shapes behavior, context engineering shapes information, and RAG grounds generation in external knowledge

Long Context Models

- RAG works well under the hypotesis you can find the right information in the VectorDB
  - Limited context window
- Brute force approach with Long Context
  - Context window 4k -> 1M+
  - = Simplicity
  - Collapsing the infrastructure - No stack
    - No need of chanking strategies, embeddings models, VectorDB, Re-Ranker, …
  - No retrival lottery - silent failure
  - No “Whole book” problem
    - No limitation of chanking the whole book

Reasons for using RAG

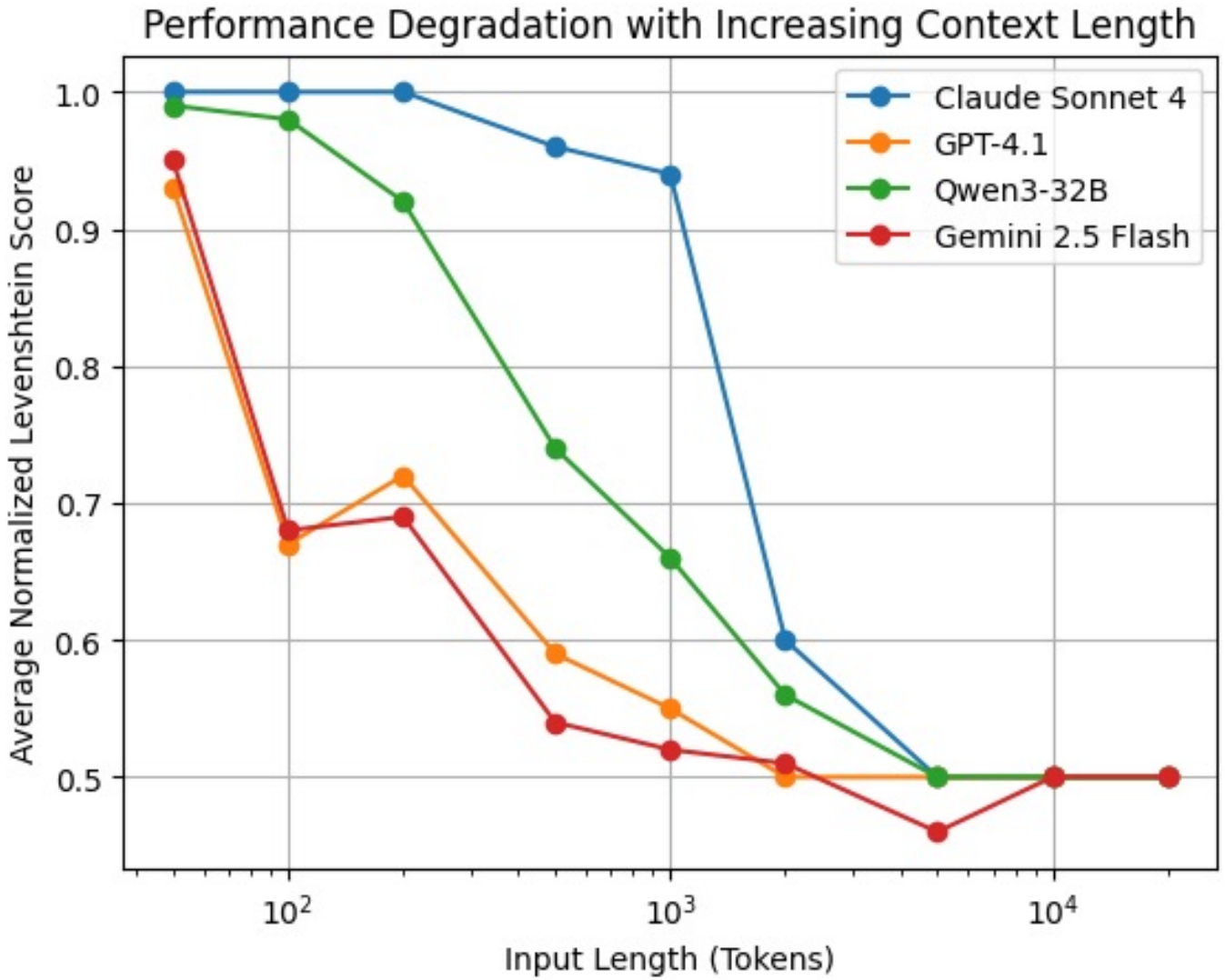
- **Re-reading TAX**
  - **Long context inefficiency**: the model must reprocess the entire document at every query
  - **High computational cost**: increased latency and resource usage
  - RAG advantage: documents are processed once and relevant chunks are reused efficiently
- **Needle in the haystack**
  - **Sparse information**: relevant detail is buried within a large amount of text
  - **Retrieval difficulty**: hard to locate the exact piece of information
  - **LLM limitation**: performance drops when searching for small details in long contexts
  - With RAG less noise is introduced (eg. Top 5 relevant chanks)
- Infinite Data Set
  - Enterprise data lake dimension (TB or PB)

Context Window of Major LLMs

Model	Context Window	Note
Gemini 3.1 Pro	**1M**	Long context leader
GPT-5.4	**1M**	Top OpenAI model
GPT-5.3 Codex	400K	Optimized for coding
GPT-5.2	400K	High performance
Claude Opus 4.6	200K	Strong reasoning
Claude Sonnet 4.6	200K	More efficient
GLM-5	200K	Emerging alternative

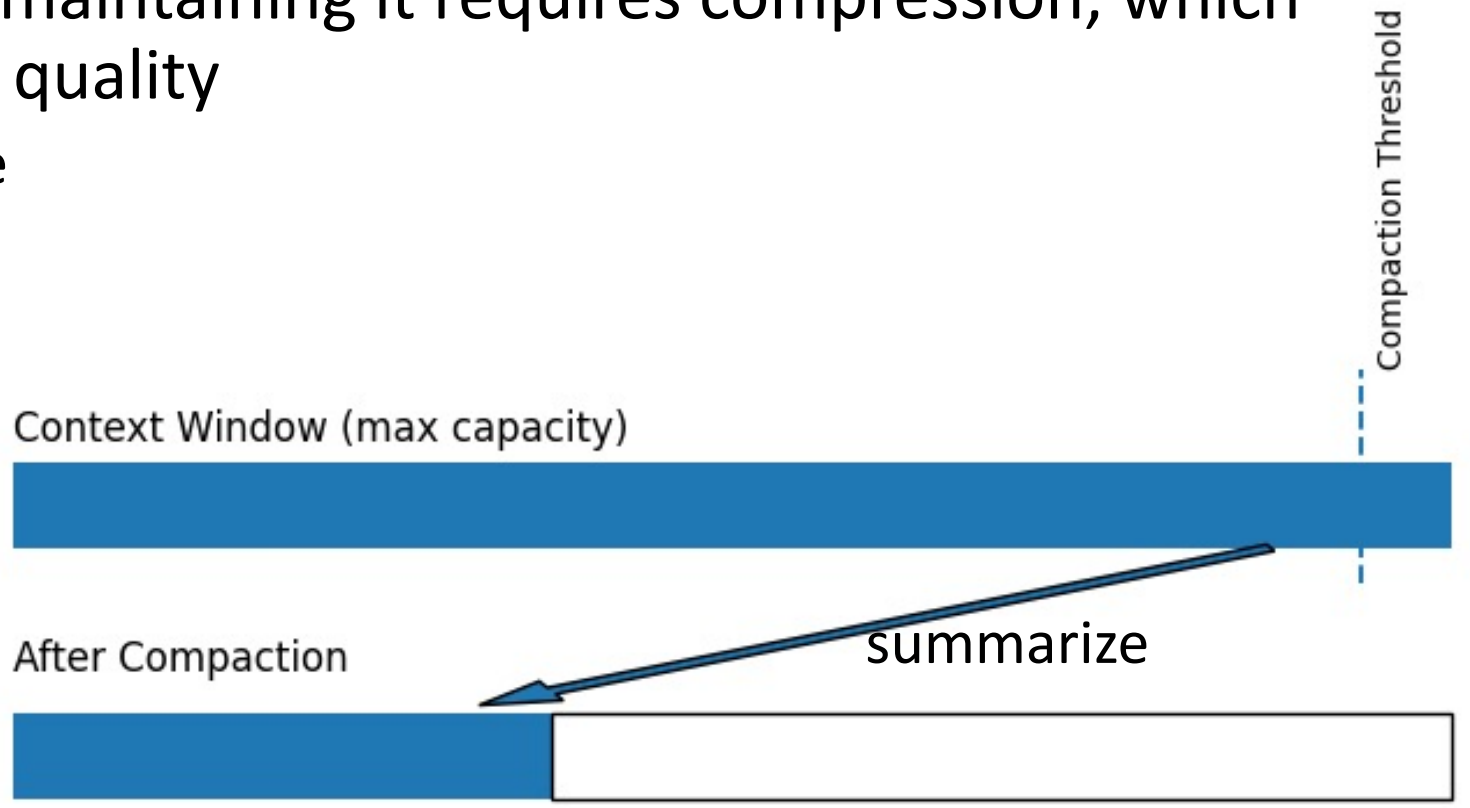
Larger context windows allow models to handle more information, but come with higher computational cost and inefficiencies

As context increases, performance degrades



Context Compaction Mechanism

- Long context is not free: maintaining it requires compression, which can degrade information quality
  - Codex, antigravity, claude



2026. MIT CSAIL, Cambridge, MA, USA

Recursive Language Models

Alex L. Zhang [1] **Tim Kraska** [1] **Omar Khattab** [1]

Abstract

We study allowing large language models (LLMs) to process arbitrarily long prompts through the lens of inference-time scaling. We propose **Recursive Language Models** (**RLM**s), a general inference paradigm that treats long prompts as part of an external *environment* and allows the LLM to *programmatically* examine, decompose, and *recursively call itself over* snippets of the prompt. We find that RLMs can successfully process inputs up to two orders of magnitude beyond model context windows and, even for shorter prompts, dramatically outperform the quality of vanilla frontier LLMs and common long-context scaffolds across four diverse long-context tasks while having comparable cost. At a small scale, we post-train the first natively recursive language model. Our model, **RLM-Qwen3-8B**, outperforms the underlying Qwen3-8B model by 28.3% on average and even approaches the quality of vanilla GPT-5 on three long-context tasks. Code is available at https://github.com/alexzhang13/rlm.

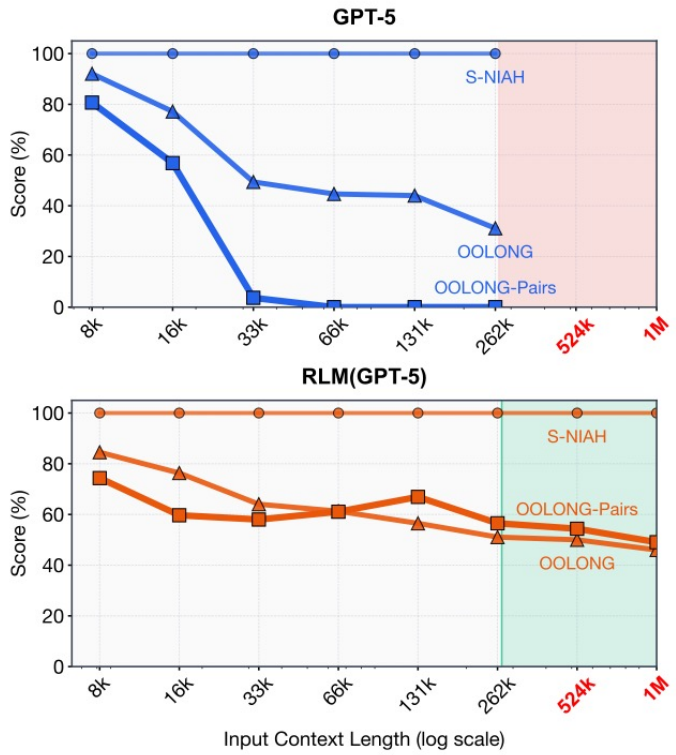
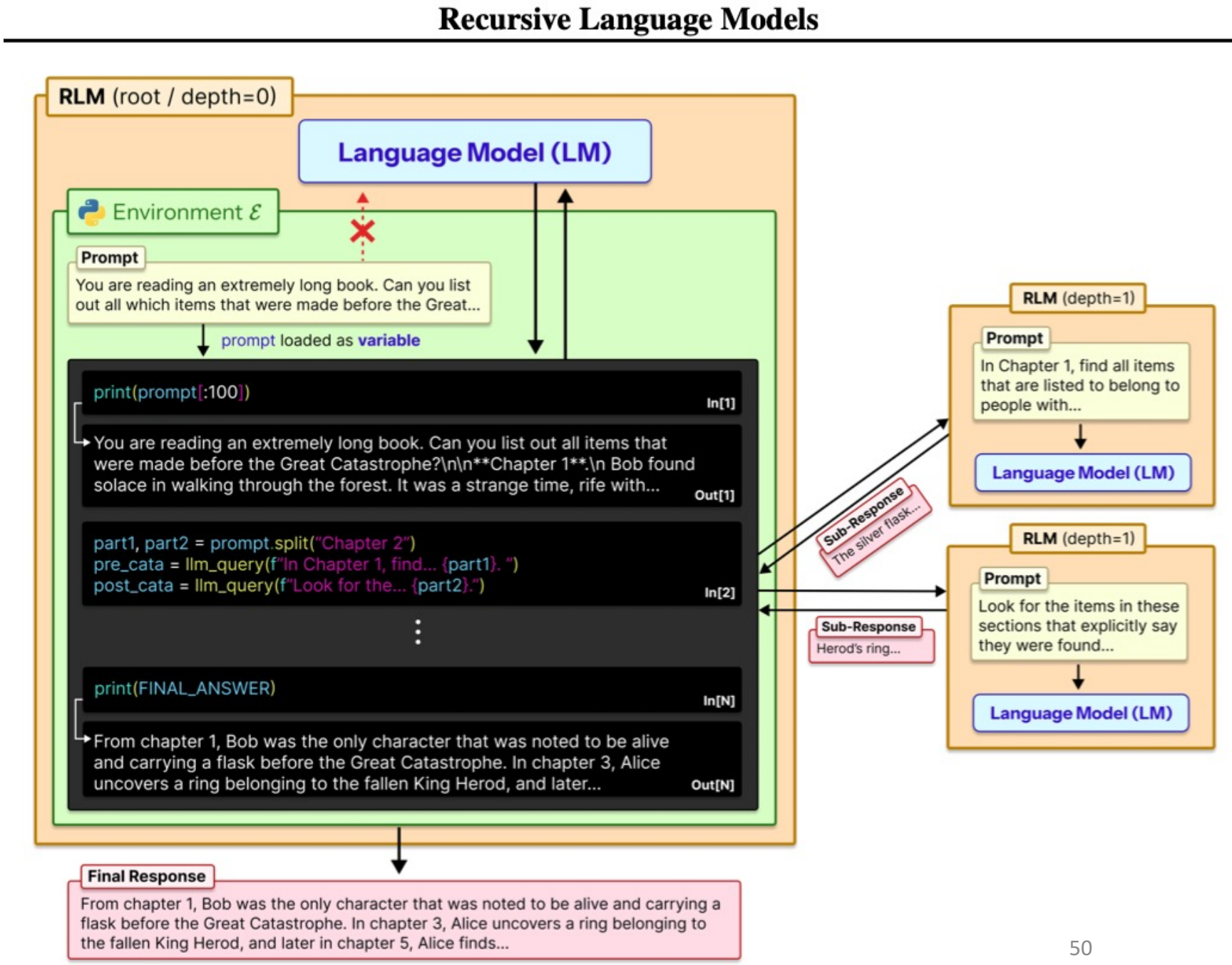
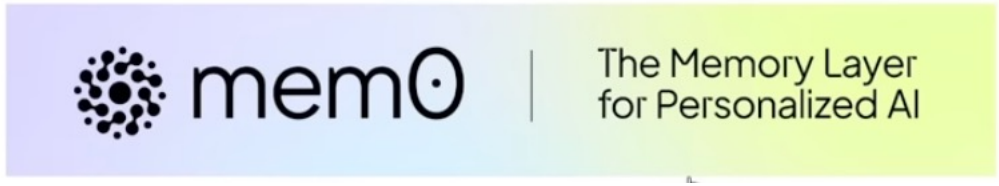


Figure 1. A comparison of GPT-5 and a corresponding RLM using GPT-5 on three long-context tasks of increasing complexity: **S-NIAH**, **OOLONG**, and **OOLONG-Pairs**. For each task, we scale the input length from 2^{13} to 2^{18}. GPT-5 performance degrades significantly as a function of both input length and task complexity, while the RLM maintains strong performance. Inputs beyond the

- The prompt is handled as a string within a Python environment
- The agent interacts with this environment by generating and executing code
- It can read portions of the document (e.g., the first 100 characters) and manipulate the text (e.g., splitting, parsing), …
- The agent is equipped with tools to operate on and transform the prompt

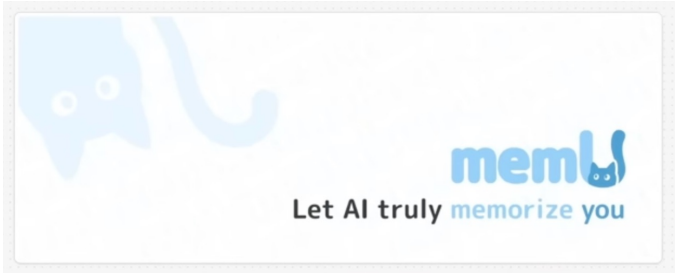


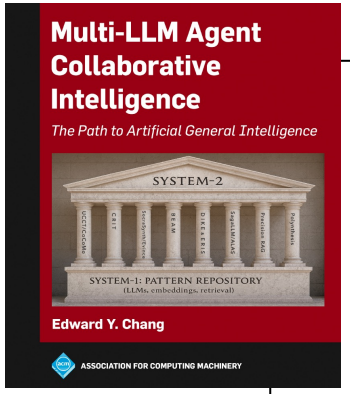
They extend the RLM for managing the agent memory

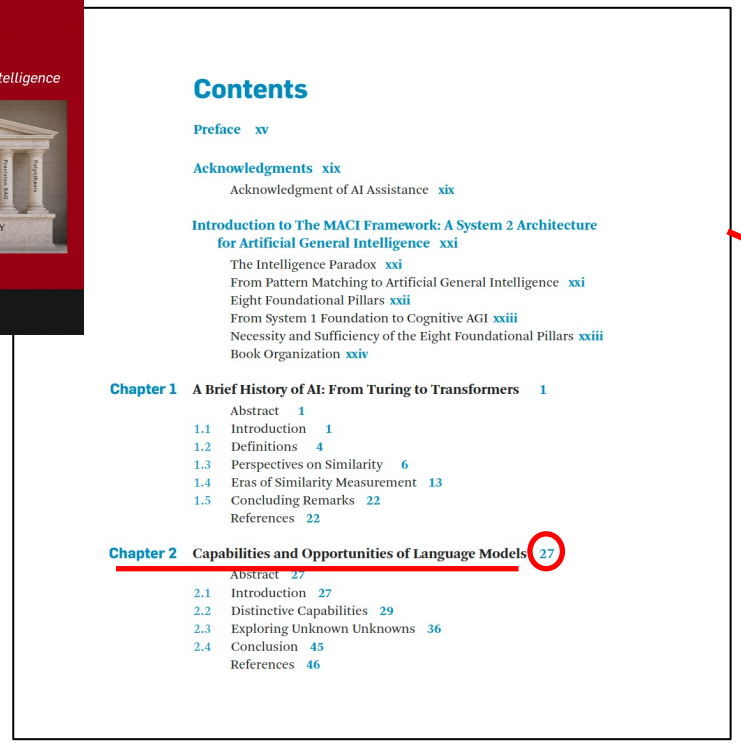












Contents

Preface xv

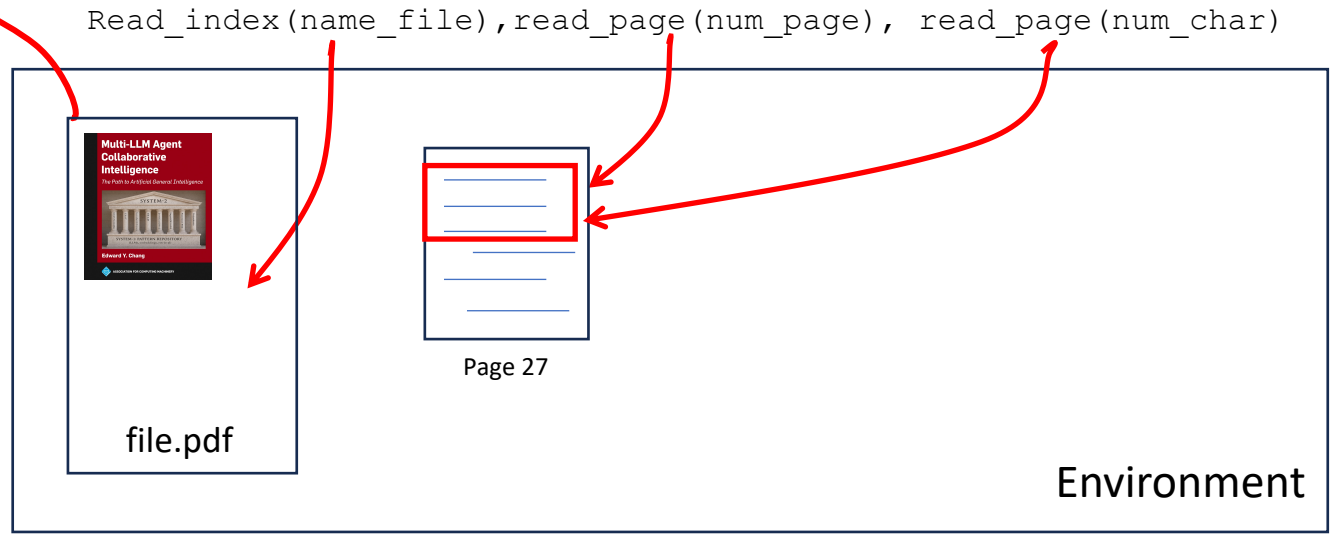
Acknowledgments xix
Acknowledgment of AI Assistance xix

Introduction to The MACI Framework: A System 2 Architecture for Artificial General Intelligence xxi
The Intelligence Paradox xxi
From Pattern Matching to Artificial General Intelligence xxi
Eight Foundational Pillars xxii
From System 1 Foundation to Cognitive AGI xxiii
Necessity and Sufficiency of the Eight Foundational Pillars xxiii
Book Organization xxiv

Chapter 1 A Brief History of AI: From Turing to Transformers 1
Abstract 1
1.1 Introduction 1
1.2 Definitions 4
1.3 Perspectives on Similarity 6
1.4 Eras of Similarity Measurement 13
1.5 Concluding Remarks 22
References 22

Chapter 2 Capabilities and Opportunities of Language Models 27
Abstract 27
2.1 Introduction 27
2.2 Distinctive Capabilities 29
2.3 Exploring Unknown Unknowns 36
2.4 Conclusion 45
References 46

598 pages



- The agent exploit the tools calling to read the document (as we might do)
- High accurancy – no semantic search

Agentic file search

Agentic file search



Features

- 🔍 6 Tools: `scan_folder` , `preview_file` , `parse_file` , `read` , `grep` , `glob`
- 📄 **Document Support**: PDF, DOCX, PPTX, XLSX, HTML, Markdown (via Docling)
- 🤖 **Powered by**: Google Gemini 3 Flash with structured JSON output
- 💰 **Cost Efficient**: ~$0.001 per query with token tracking
- 🌐 **Web UI**: Real-time WebSocket streaming interface
- 📊 **Citations**: Answers include source references

Memu



Memory as File System, File System as Memory

memU treats **memory like a file system**—structured, hierarchical, and instantly accessible.

File System	memU Memory
Folders	Categories (auto-organized topics)
Files	Memory Items (extracted facts, preferences, skills)
Symlinks	Cross-references (related memories linked)
Mount points	Resources (conversations, documents, images)

Why this matters:

- **Navigate memories** like browsing directories—drill down from broad categories to specific facts
- **Mount new knowledge** instantly—conversations and documents become queryable memory
- **Cross-link everything**—memories reference each other, building a connected knowledge graph
- **Persistent & portable**—export, backup, and transfer memory like files

```
memory/
├── preferences/
│   ├── communication_style.md
│   └── topic_interests.md
├── relationships/
│   ├── contacts/
│   └── interaction_history/
├── knowledge/
│   ├── domain_expertise/
│   └── learned_skills/
└── context/
    ├── recent_conversations/
    └── pending_tasks/
```

OpenClaw

Bootstrapping

Bootstrapping

Sessions and memory

Session Management

Session Pruning

Session Tools

Memory

Compaction

Multi-agent

Multi-Agent Routing

Presence

Messages and delivery

Sessions and memory

Memory

OpenClaw memory is **plain Markdown in the agent workspace**. The files are the source of truth; the model only "remembers" what gets written to disk.

Memory search tools are provided by the active memory plugin (default: `memory-core`). Disable memory plugins with `plugins.slots.memory = "none"`.

Memory files (Markdown)

The default workspace layout uses two memory layers:

- `memory/YYYY-MM-DD.md`

OpenClaw

Memory files (Markdown)

The default workspace layout uses two memory layers:

- `memory/YYYY-MM-DD.md`
  - Daily log (append-only).
  - Read today + yesterday at session start.
- `MEMORY.md` (optional)
  - Curated long-term memory.

Memory tools

OpenClaw exposes two agent-facing tools for these Markdown files:

- `memory_search` — semantic recall over indexed snippets.
- `memory_get` — targeted read of a specific Markdown file/line range.

OpenClaw

When to write memory

- Decisions, preferences, and durable facts go to `MEMORY.md` .
- Day-to-day notes and running context go to `memory/YYYY-MM-DD.md` .
- If someone says “remember this,” write it down (do not keep it in RAM).
- This area is still evolving. It helps to remind the model to store memories; it will know what to do.
- If you want something to stick, **ask the bot to write it** into memory.

OpenClaw

Automatic memory flush (pre-compaction ping)

When a session is **close to auto-compaction**, OpenClaw triggers a **silent, agentic turn** that reminds the model to write durable memory **before** the context is compacted. The default prompts explicitly say the model *may reply*, but usually `NO_REPLY` is the correct response so the user never sees this turn.

Vector memory search

OpenClaw can build a small vector index over `MEMORY.md` and `memory/*.md` so semantic queries can find related notes even when wording differs.

The Agentic Search mechanism is used for **agentic coding** to manage the progress of coding activities

